

very popular and flexible open source ORM solution). Thanks to the dynamic nature of Groovy, with its static and dynamic typing, along with the convention of Grails, there is much less configuration involved in creating Grails domain classes. While this is a topic that requires a separate article and a lot of practice, I will try to cover the main points about GORM. Grails is known as a domain driven language, which means that an application can be built in a bottom-up manner. Domain classes are the core of any business application. They can be linked together through relationships which can be one-to-one, one-to-many or many-to-many. By default, Grails applications are configured to use an in-memory database called HSQLDB. For the purposes of this tutorial, I will walk you through setting up MySQL. To do so, follow the steps listed below.

1. *Creating a database in MySQL:* Use the following command in MySQL to create a database.

```
CREATE DATABASE test_prod
```

This will create a database called *test_prod*, which we will use in production. Additionally, we can create a database called *test_dev*, which can be used in development mode for testing our application.

2. *Updating the BuildConfig.groovy file in grails-app/conf:* We do this to tell Grails that our application needs to use the MySQL connector, so we need to add that as a dependency to our application. All dependencies must be specified in this file. When your application is run, all dependencies specified here are automatically downloaded and installed so that your application can use them. Open up the *BuildConfig* file located in the *grails-app/conf* folder and find the line...

```
// runtime 'mysql:mysql-connector-java:5.1.24'
```

...and uncomment it. The complete *BuildConfig* file can be downloaded from http://www.opensourceforu.com/article_source_code/sept14/grails.zip

3. *Updating DataSource.groovy in grails-app/conf:* After setting up the dependencies, we must set up our application to use certain data sources. In our project, we'd like it to use our *test_dev* for development purposes and *test_prod* for production. First, we must edit the *DataSource* variable so that it knows how to use the *InnoDB* engine, as follows:

```
dataSource {
    pooled = true
    driverClassName = "com.mysql.jdbc.Driver"
    dialect = "org.hibernate.dialect.MySQL5InnoDBDialect"
}
```

The next step is to configure the environments variable to use *test_dev* for development and *test_prod* for production.

Edit your environment's variable as shown in the code snippet below.

```
environments {
    development {
        dataSource {
            dbCreate = "create-drop" // one of 'create',
            'create-drop', 'update', 'validate', ''
            url = "jdbc:mysql://localhost/test_dev"
            username = "root"
            password = "root"
        }
    }
    test {
        dataSource {
            dbCreate = "update"
            url = "jdbc:mysql://localhost/test_prod"
            username = "root"
            password = "root"
        }
    }
    production {
        dataSource {
            dbCreate = "update"
            url = "jdbc:mysql://localhost/test_prod"
            username = "root"
            password = "root"
        }
    }
}
```

dbCreate deserves a special mention. It determines what happens to your database between server restarts. For example, a value of *create-drop* means that the database tables will be dropped and then recreated when you start the server, so you lose any data that was in there. This is fine for testing, but is definitely not what you want in production.

4. *Creating a new domain class:* We're done with setting up Grails to use MySQL; so now we can simply create a domain class which will be persisted in our database. Let's create a domain class that represents a user of our application, using the following command.

```
grails create-domain-class test.User
```

After this, Grails will tell you that it created a few files. Let's now edit Grails *app/domain/test/User.groovy* file to store the information of our users. The following is the edited *User.groovy* file.

```
package test
```